

Project: AWS Cost Monitoring & Budget Alerts

Services: AWS Budgets, Cost Explorer, Lambda, SES.

Goal: Automatically notify when budget thresholds are met.

Skills: Billing alerts, email notifications, tagging strategies.

Definition: AWS Cost Monitoring & Budget Alerts is a serverless cloud cost management project that continuously tracks AWS service usage and spending using Cost Explorer and AWS Budgets. The project automatically detects when predefined budget thresholds are exceeded and triggers a Lambda function to send real-time email notifications via Amazon SES, helping users prevent unexpected cloud expenses and maintain effective cost control.

Scope of the Project:

- Implement automated monitoring of AWS service usage and spending.
- Detect and alert users when predefined budget thresholds are exceeded.
- Send real-time email notifications via Amazon SES for actionable alerts.
- Utilize Lambda for automation and customizable alert logic.
- Demonstrate practical skills in serverless architecture, budget management, and cloud cost optimization.

Methodology:

1. Budget-Triggered Serverless Alert Method

- Uses AWS Budgets, Cost Explorer, Lambda, and Amazon SES.
- Automatically detects budget threshold breaches.
- Triggers Lambda when spending exceeds defined limits.
- Sends email alerts via SES in near real time.
- Fully event-driven with minimal manual intervention.
- Cost-efficient and safe within AWS Free Tier.
- Ideal for cost governance and learning FinOps basics.

2: Scheduled Cost Evaluation Method

- Uses Cost Explorer, Lambda, and Amazon SES.
- Lambda runs on a schedule to check cost data.
- Implements custom cost comparison logic in code.
- Sends email alerts via SES when limits are crossed.
- Offers greater flexibility for custom analysis.
- Slightly more complex than managed budget alerts.
- Suitable for advanced monitoring and experimentation.

AWS Services used in the Project:

1. AWS Cost Explorer

- **Definition:** A tool to visualize, analyze, and understand AWS costs and usage over time.
- **Purpose:** To provide detailed cost and usage data for monitoring spending patterns.
- **Role in Project:** Supplies cost and usage information to determine if budget thresholds are being approached or exceeded.

2.AWS Budgets

- **Definition:** A service that allows setting cost, usage, and reservation budgets with alerts.
- **Purpose:** To define spending limits and notify when thresholds are crossed.
- **Role in Project:** Monitors AWS spending and triggers Lambda automatically when budget thresholds are exceeded.

3.AWS Lambda

- **Definition:** A serverless compute service that runs code in response to events without provisioning servers.
- **Purpose:** To automate processes and perform tasks triggered by events.
- **Role in Project:** Processes budget alert events and generates customized messages for email notifications.

4.Amazon SES (Simple Email Service)

- **Definition:** A cloud-based email sending service for notifications and communication.
- **Purpose:** To deliver automated email notifications.
- **Role in Project:** Sends real-time email alerts to users when Lambda is triggered, ensuring timely communication of cost issues.

Architecture Diagram:



Project Implementation Steps:

STEP 1: Enable Cost Explorer

- Log in to your AWS Management Console.
- Navigate to Billing → Cost Explorer.
- Click Enable Cost Explorer.
- Wait 24 hours for cost data to populate.

The screenshot shows the AWS Cost Explorer console. The left sidebar contains navigation links for Billing and Cost Management, including Billing View, Home, Getting Started, Dashboards, Billing and Payments, and Cost and Usage Analysis. The main content area displays a 'New cost and usage report' with a 'Cost and usage overview' section showing Total cost, Average monthly cost, and Service count. Below this is a 'Cost and usage graph' showing costs over time. The right sidebar contains 'Report parameters' including Time (Date Range, Granularity), Group by (Dimension, Service), and Filters (Applied filters, Service, Linked account).

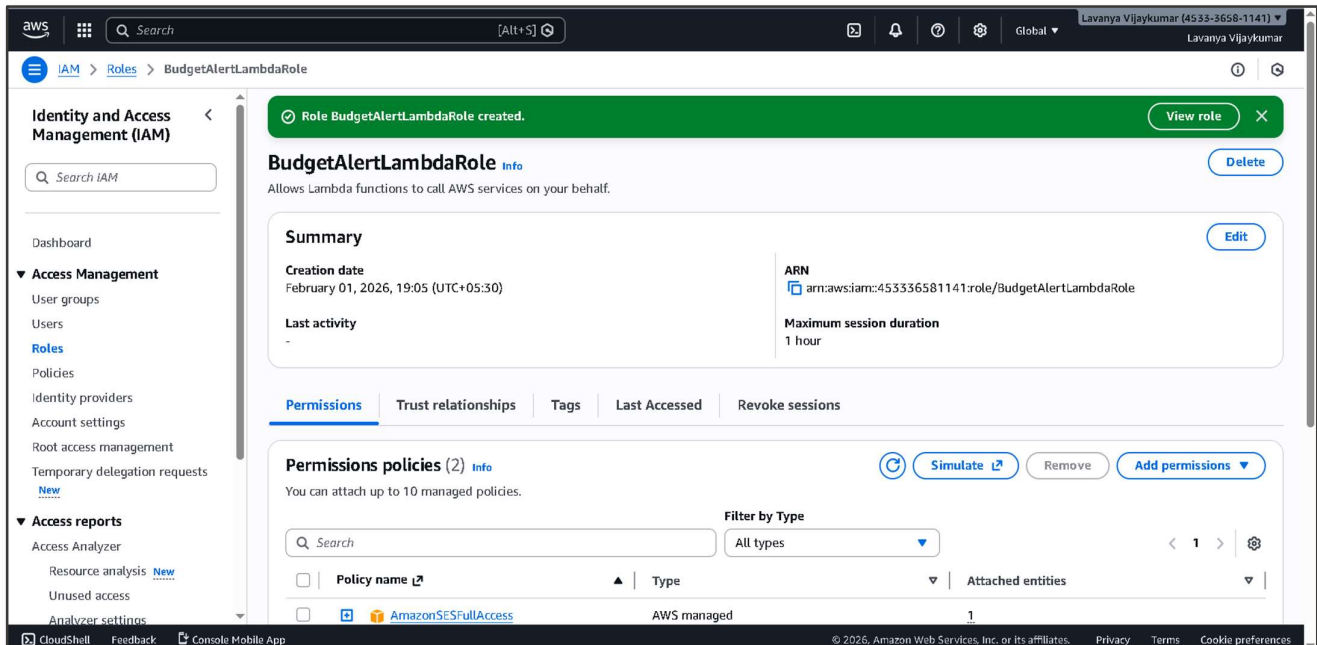
STEP 2: Verify Email in SES

- Open Amazon SES
- Go to Verified Identities → Create Identity
- Choose Email Address
- Enter your email ID
- Click Create Identity
- Open your inbox and confirm verification

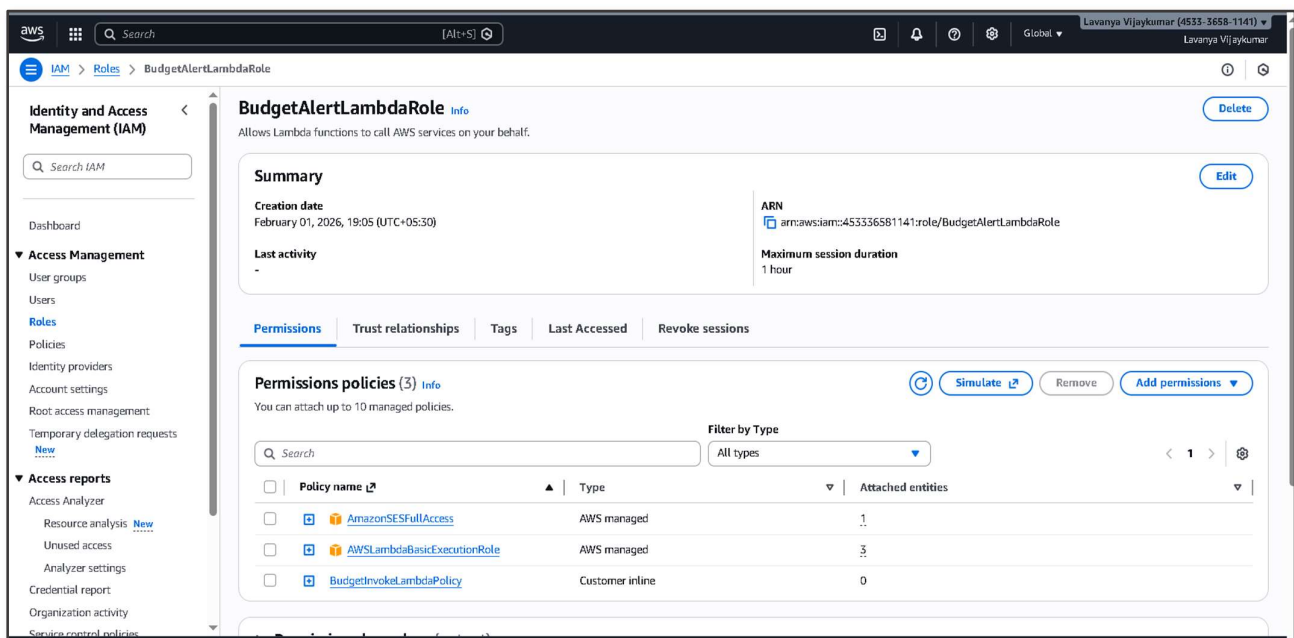
The screenshot shows the Amazon SES console for the identity 'lavulavanya356@gmail.com'. The left sidebar contains navigation links for Amazon SES, including Get set up, Account dashboard, Reputation metrics, SMTP settings, What's new, Configuration, Identities, Configuration sets, Tenants, Dedicated IPs, Global endpoints, Email templates, Suppression list, Email receiving, Email validation, Overview, Email addresses validation, Auto Validation, and Dashboard. The main content area displays the 'Summary' section with Identity status (Verified), Amazon Resource Name (ARN), and AWS Region (Asia Pacific (Mumbai)). Below this is a 'Recommendations' section with a message about recommendations for verified domain identities. The bottom section shows 'DomainKeys Identified Mail (DKIM)' configuration.

STEP 3: Create IAM Role for Budget to Invoke Lambda

- Go to IAM → Roles → Create role
- Trusted entity type: AWS Service
- Use case: Budgets
- Click Next
- Attach Policies:
 1. AWSLambdaBasicExecutionRole
 2. AmazonSESEFullAccess
- Role name: BudgetAlertLambdaRole
- Create role

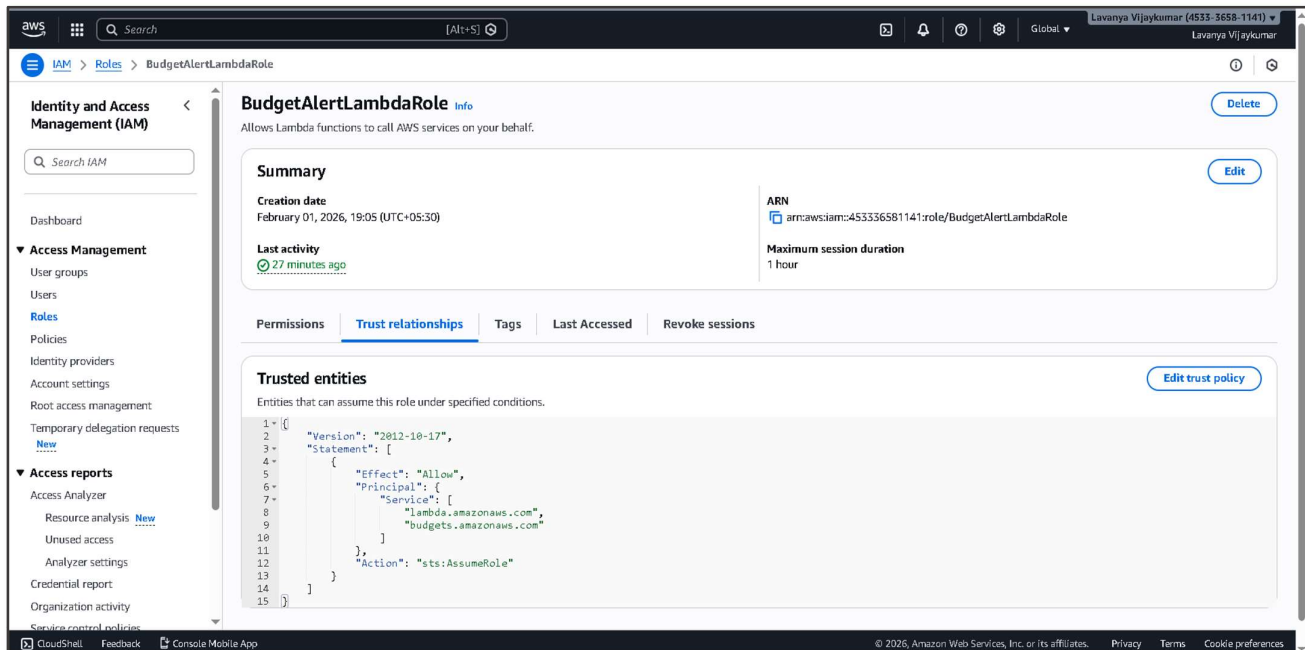


- Attach Inline policy: BudgetInvokeLambdaPolicy



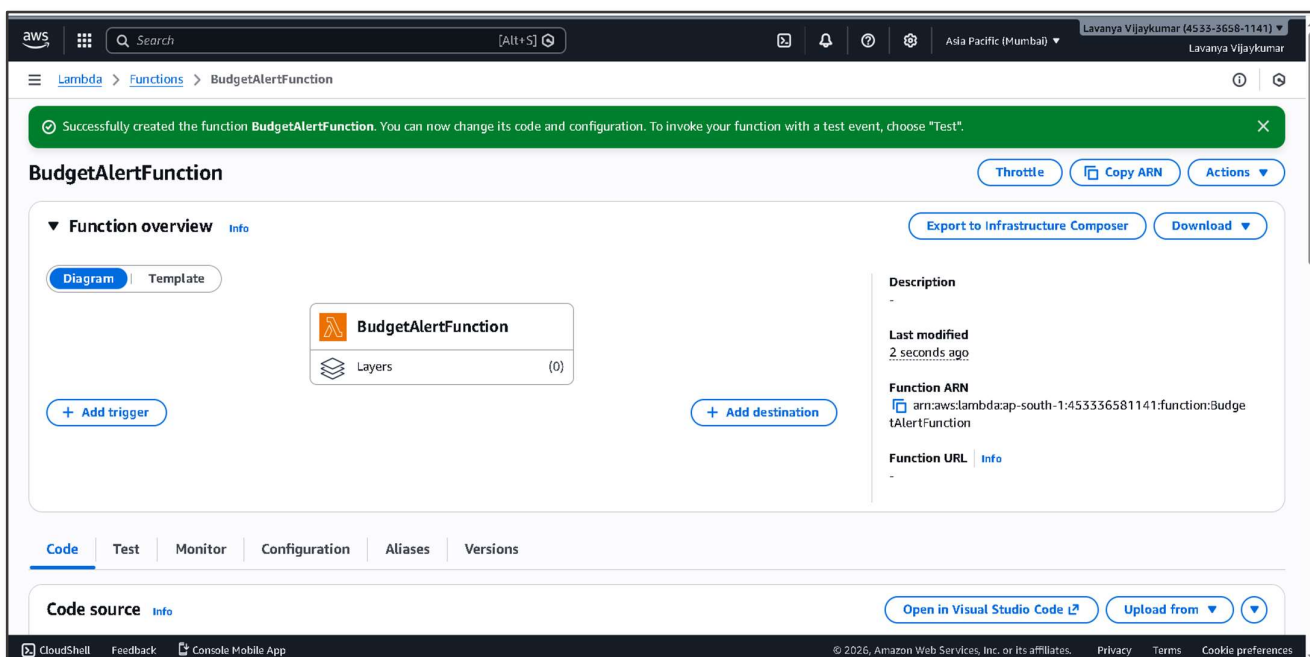
STEP 4: Update IAM Trust Relationship

- Open IAM → Roles → BudgetAlertLambdaRole
- Trust relationships → Edit
- Save changes



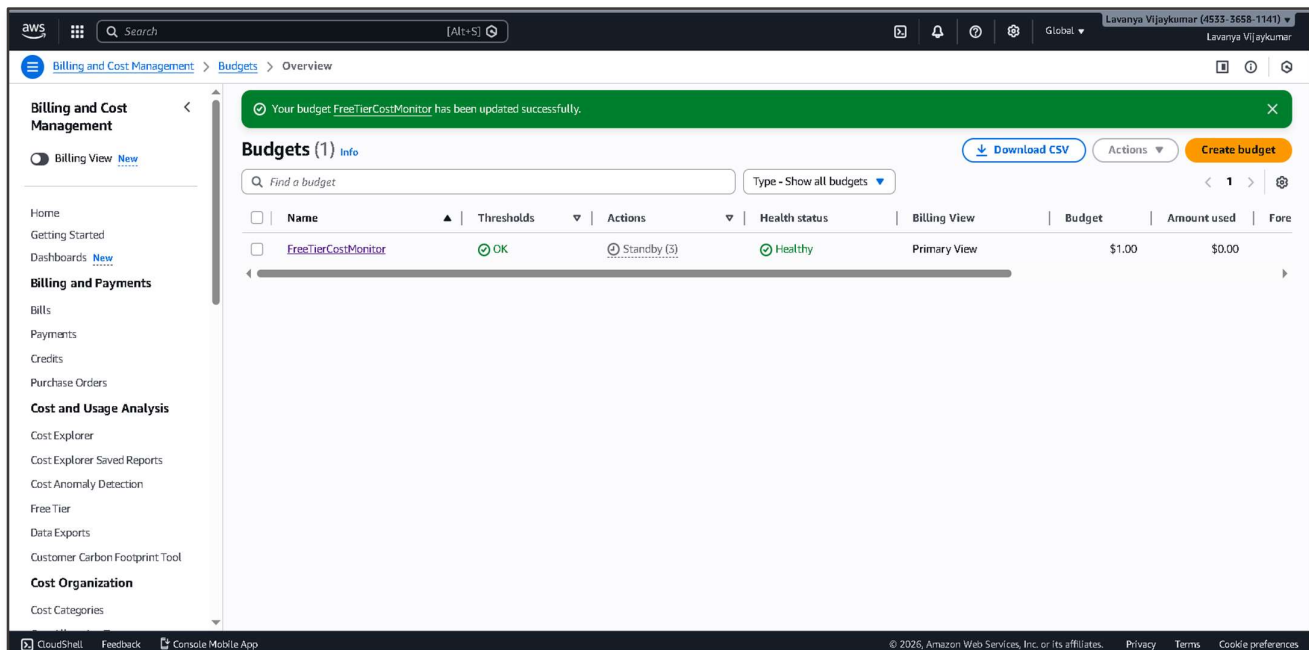
STEP 5: Create Lambda Function

- Go to AWS Lambda → Create Function → Author from Scratch
- Function Name: BudgetAlertFunction
- Runtime: Python 3.10
- Role: BudgetAlertLambdaRole
- Click Create Function



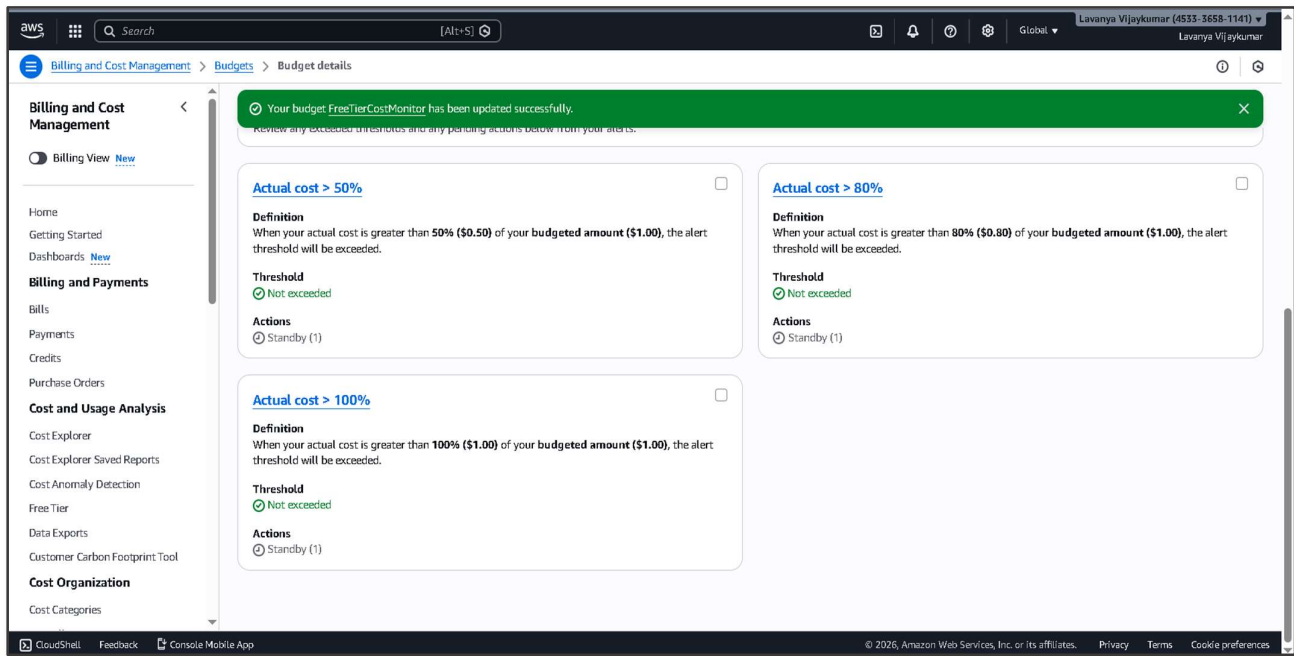
STEP 6: Create a Budget

- Go to Billing → Budgets → Create Budget
- Budget Type: Cost budget
- Budget Name: FreeTierCostMonitorTest
- Budget Renewal Type: Monthly
- Start Month: e.g., Feb 2026
- Budgeting Method: Fixed
- Budget Amount: \$1
- Scope: All AWS services
- Click Configure Alerts while creating the budget
- Add alert thresholds one by one
 1. 50%
 2. 80%
 3. 100%
- Alert Type: Actual
- Send to: Email (verified in SES) OR Lambda function



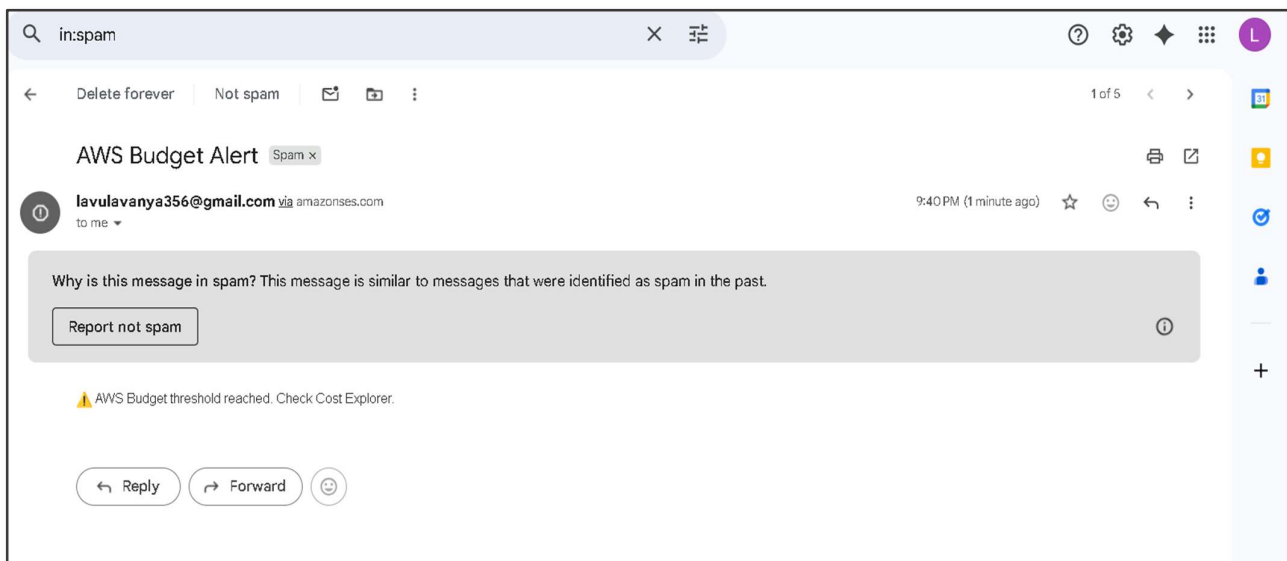
STEP 7: Add Budget Action (IMPORTANT STEP)

- For each threshold:
- Click Add action
- Action type: IAM Role
- Select IAM role: BudgetAlertLambdaRole
- Which action type should be applied?
 1. Select IAM Policy
 2. Choose the auto-generated Lambda invoke policy (example: LambdaInvokeScopedAccessPolicy-xxxx)
- Save



STEP 8: Email Notification

Receive automatic notification alert via email when the AWS budget threshold is reached.



Troubleshooting:

Issue: Action failed to save when adding a Lambda invocation to the budget.

Cause: The IAM role attached to the Lambda (BudgetAlertLambdaRole) does not have the correct trust relationship, so AWS Budgets cannot assume the role to invoke the function.

Solution: Edit the IAM role's trust relationship to allow AWS Budgets to assume it.

- In the IAM console, go to the role → Trust Relationships → Edit trust relationship
- Add budgets.amazonaws.com as a trusted service.
- Save the changes and then re-add the action in your budget.

Conclusion:

The AWS Cost Monitoring & Budget Alerts project successfully demonstrates how to track AWS usage, set budget thresholds, and receive notifications when spending limits are reached. By integrating AWS Budgets, Lambda, and SES, the project provides an automated mechanism to alert users of potential overspending, even in a Free Tier environment.

Through this project, key skills such as billing alerts, email notifications, and cost monitoring strategies were applied. The project also highlights the importance of IAM roles and permissions for secure automation and demonstrates how simulated usage can be used to test workflows without incurring real costs.

Overall, this project emphasizes proactive cost management in AWS, ensuring users can monitor and control their cloud expenses effectively while gaining hands-on experience with serverless automation and notification workflows.